Take Two

Take Two is a rehearsal coach for anyone who needs to land an idea out loud: a conference talk, a thesis defense, a wedding toast, a startup pitch, a slam poem. You mark your script with what you're going for (slow down here, pause there, explain this), rehearse out loud, and it shows where your delivery matched those marks and where it didn't.

The problem

PowerPoint's Speaker Coach, Yoodli and Orai compare everyone to one idea of a good speaker: steady pace, few fillers, no dead air. They know nothing about your speech, so they can't tell a deliberate pause from losing your place. But a deliberate pause is the whole point of a toast's punchline, a poem's line break, or the sentence a talk exists to deliver. And nobody can hear their own pace while they're speaking.

Who it's for

- **Technical and academic talks:** the key result said slower, then silence; terms explained before the room needs them.
- **Celebratory speeches** (toasts, tributes): the pause after the punchline for the room to react; a sincere last line, not rushed.
- **Performance poetry and slam:** the breath at each line break, the drop before the last line.
- **Pitches and interviews:** the number and the ask said slowly enough to be heard, inside a hard time limit.

Why voice

What Take Two checks only exists in how you say things: pace, pauses, timing. A transcript loses all of it, and you can't notice it yourself while you're reading, holding a clicker or a glass, and focused on the words. So the app does the listening, and talks back: Q&A after a talk is oral and unscripted, so an examiner asks aloud while your hands stay on your notes. Marking a script and reading results are easier with a keyboard and screen, so those stay that way.

How it works

Marks describe intent, not a style, so the same few serve a lecture and a toast: a time budget per section, `[KEY]` for a line to slow down on, `/` and `//` for short and long pauses, and `[DEFINE: term]` for anything the audience may not know (a technical term, a family in-joke). After a take you get your script back with each mark labelled met, close or diverged, the measurement ("19% faster than your median"), and a click to hear that moment. No score. **Hear it** plays a line as you marked it, or a coach's version, each cue labelled checked or demonstration only; then you say it and it is measured. A **spoken examiner** asks Q&A questions aloud, records each answer hands-free, and asks at most one follow-up built on your words. You can also drill a line, compare takes, or speak unscripted.

How it's built

FastAPI and TypeScript, running locally. faster-whisper transcribes with word timestamps; Silero's voice activity detector measures pauses from the audio. Pace is compared to your own median, not a fixed target. Audio stays on your computer unless you add a cloud key. The default voice is Kokoro-82M, a neural voice running locally (the browser's voice is the fallback). 371 Python unit tests, 25 frontend tests, a synthetic recording with known pauses through the full pipeline, and browser tests that record through Chrome's fake microphone.

AI tools: what I used and how

I built it with Claude Code, in milestones, from a spec and product rules I wrote first (commits M1 to M18). In the app, Whisper and Silero handle audio. Claude handles optional features: suggesting marks, checking whether a term was explained, coaching notes, likely audience questions, the coach's version of a line, proposed pronunciations, and the examiner's follow-ups. It sees only text and numbers, never audio, and code checks its output, for example dropping any quote that isn't in the transcript. These have only been tested against a mocked model so far.

What's next

Testing with real speakers of each kind, a first run against the real model, and listening to the voice features in a real room.